

Python Code Used In The Website :

```
import os
import re
import pandas as pd
import pdfplumber
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch_geometric.nn import SAGEConv
import networkx as nx
import numpy as np

# =====
# 1. UNIVERSAL MULTI-FORMAT FILE PARSER
# =====
class UniversalFileLoader:
    """Detects file format and extracts transactions with actual PDF Transaction IDs."""

    @staticmethod
    def load_any_file(file_path):
        if not os.path.exists(file_path):
            raise FileNotFoundError(f"File not found: {file_path}")

        ext = os.path.splitext(file_path)[-1].lower()
        print(f"📁 Reading input file ({ext}): {file_path}")

        if ext in ['.xlsx', '.xls']:
            df = pd.read_excel(file_path)
            return UniversalFileLoader._standardize_df(df)
        elif ext == '.csv':
            df = pd.read_csv(file_path)
            return UniversalFileLoader._standardize_df(df)
        elif ext == '.pdf':
            return UniversalFileLoader._extract_from_pdf(file_path)
        elif ext in ['.png', '.jpg', '.jpeg']:
            return UniversalFileLoader._extract_from_image(file_path)
        else:
            raise ValueError(f"Unsupported file format: {ext}")

    @staticmethod
    def _standardize_df(df):
        if df.empty:
            return pd.DataFrame(columns=['tx_id', 'source', 'target', 'amount'])

        df.columns = [str(c).strip().lower() for c in df.columns]

        debit_col = next((c for c in df.columns if 'debit' in c or 'wdl' in c or 'dr' in c), None)
        credit_col = next((c for c in df.columns if 'credit' in c or 'dep' in c or 'cr' in c), None)

        if debit_col and credit_col and 'amount' not in df.columns:
            def combine_debit_credit(row):
                d = re.sub(r'^\d.', '', str(row[debit_col])) if pd.notna(row[debit_col]) else ''
                c = re.sub(r'^\d.', '', str(row[credit_col])) if pd.notna(row[credit_col]) else ''
                if d and d != '-':
                    return -float(d)
                elif c and c != '-':
                    return float(c)
            df['amount'] = df.apply(combine_debit_credit, axis=1)
```

```

        return 0.0

    df['amount'] = df.apply(combine_debit_credit, axis=1)

    rename_map = {}
    for col in df.columns:
        if any(k in col for k in ['tx_id', 'txid', 'ref', 'chq', 'cheque', 'reference', 'txn id', 'tran id']):
            rename_map[col] = 'tx_id'
        elif any(k in col for k in ['details', 'narration', 'description', 'source', 'from', 'sender', 'src']):
            rename_map[col] = 'source'
        elif any(k in col for k in ['target', 'to', 'receiver', 'dst']):
            rename_map[col] = 'target'
        elif any(k in col for k in ['amount', 'value', 'sum', 'amt', 'usd']) and col != 'amount':
            rename_map[col] = 'amount'

    df.rename(columns=rename_map, inplace=True)

    if 'target' not in df.columns:
        df['target'] = 'SBI_MAIN_ACC'

    return UniversalFileLoader._clean_columns(df)

@staticmethod
def _extract_from_pdf(file_path):
    """Extracts transactions along with actual Transaction/Ref IDs from PDF."""
    records = []

    with pdfplumber.open(file_path) as pdf:
        for page in pdf.pages:
            tables = page.extract_tables()
            for table in tables:
                for row in table:
                    if not row or len(row) < 4:
                        continue

                    cleaned = [str(cell).replace('\n', ' ').strip() if cell else '' for cell in row]
                    row_str = " ".join(cleaned)

                    if 'Value Date' in row_str or 'Post Date' in row_str or 'STATEMENT OF ACCOUNT'
in row_str:
                        continue

                    details = cleaned[2] if len(cleaned) > 2 else row_str
                    debit = cleaned[-3] if len(cleaned) >= 6 else ''
                    credit = cleaned[-2] if len(cleaned) >= 6 else ''

                    debit_val = re.sub(r'^\d.', '', debit)
                    credit_val = re.sub(r'^\d.', '', credit)

                    amt = 0.0
                    if debit_val and debit_val != '.':
                        amt = -float(debit_val)
                    elif credit_val and credit_val != '.':
                        amt = float(credit_val)

                    if details or amt != 0.0:
                        # Extract Transaction ID / Reference Number from SBI Narration
                        # Look for UPI RRR/Txn ID or Cheque/Ref numbers
                        ref_match = re.search(r'UPI/(?:(DR|CR)/(\d+))|TRANSFER-[^/]+/(\d+)|(TRANSFER
TO \d+)|CHQ NO (\d+)', details)

```

```

        if ref_match:
            tx_id = next(g for g in ref_match.groups() if g is not None)
        elif len(cleaned) > 3 and cleaned[3].replace(' ', '').isalnum() and cleaned[3] != '':
            tx_id = cleaned[3]
        else:
            tx_id = "N/A"

        # Parse counterparty name from UPI string
        party_match = re.search(r'UPI/(?:(DR|CR)/\d+/?([^\s/]+)', details)
        party = party_match.group(1) if party_match else details

        records.append({
            'tx_id': tx_id,
            'source': party if amt < 0 else "SBI_ACCOUNT",
            'target': "SBI_ACCOUNT" if amt < 0 else party,
            'amount': amt
        })

    if not records:
        page_text = page.extract_text()
        if page_text:
            parsed_df = UniversalFileLoader._parse_raw_text(page_text)
            if not parsed_df.empty:
                records.extend(parsed_df.to_dict('records'))

df = pd.DataFrame(records)
return UniversalFileLoader._clean_columns(df)

@staticmethod
def _extract_from_image(file_path):
    try:
        import pytesseract
        from PIL import Image

        img = Image.open(file_path)
        text = pytesseract.image_to_string(img)

        if not text.strip():
            raise ValueError("OCR scanning resulted in empty text.")

        return UniversalFileLoader._parse_raw_text(text)
    except ImportError:
        raise RuntimeError("pytesseract and tesseract-ocr are required for image scanning.")

@staticmethod
def _parse_raw_text(text):
    records = []
    lines = text.split('\n')

    date_pattern = re.compile(r'\d{2}/\d{2}/\d{4}\d{2}-\d{2}-\d{4}')
    amount_pattern = re.compile(r'(\d{1,3}(?:,\d{3})*\.\d{2})')

    current_record = None

    for line in lines:
        line_str = line.strip()
        if not line_str:
            continue

        if date_pattern.search(line_str):

```

```

        if current_record:
            records.append(current_record)

        amounts = amount_pattern.findall(line_str)
        amt = 0.0
        if len(amounts) >= 2:
            amt = float(amounts[-2].replace(',', ''))
            if "WDL" in line_str or "DR" in line_str:
                amt = -amt
        elif len(amounts) == 1:
            amt = float(amounts[0].replace(',', ''))
            if "WDL" in line_str or "DR" in line_str:
                amt = -amt

        # Extract Txn/Ref ID
        ref_match = re.search(r'UPI/(?DR|CR)/(\d+)|TRANSFER-[^/]+/(\d+)|(TRANSFER TO \d+)',
line_str)
        tx_id = next((g for g in ref_match.groups() if g is not None), "N/A") if ref_match else "N/
A"

        party_match = re.search(r'UPI/(?DR|CR)/\d+/([^\s/]+)', line_str)
        party = party_match.group(1) if party_match else line_str

        current_record = {
            'tx_id': tx_id,
            'source': party if amt < 0 else "SBI_ACCOUNT",
            'target': "SBI_ACCOUNT" if amt < 0 else party,
            'amount': amt
        }
    elif current_record:
        current_record['source'] += " " + line_str

    if current_record:
        records.append(current_record)

df = pd.DataFrame(records)
return UniversalFileLoader._clean_columns(df)

@staticmethod
def _clean_columns(df):
    if df.empty or len(df.columns) == 0:
        df = pd.DataFrame(columns=['tx_id', 'source', 'target', 'amount'])

    for col in ['tx_id', 'source', 'target', 'amount']:
        if col not in df.columns:
            df[col] = "UNKNOWN" if col != 'amount' else 0.0

    df['amount'] = df['amount'].astype(str).str.replace(r'[\$,]', '', regex=True)
    df['amount'] = pd.to_numeric(df['amount'], errors='coerce').fillna(0.0)

    df['tx_id'] = df['tx_id'].astype(str).str.strip()
    df['source'] = df['source'].astype(str).apply(
        lambda x: f"ACC_{int(x):04d}" if x.isdigit() else x.strip().upper()
    )
    df['target'] = df['target'].astype(str).apply(
        lambda x: f"ACC_{int(x):04d}" if x.isdigit() else x.strip().upper()
    )

    return df[['tx_id', 'source', 'target', 'amount']]

```

```

# =====
# 2. GRAPH SAGE NEURAL NETWORK MODEL
# =====
class FraudGNN(nn.Module):
    def __init__(self, in_channels, hidden_channels, out_channels):
        super(FraudGNN, self).__init__()
        self.conv1 = SAGEConv(in_channels, hidden_channels)
        self.conv2 = SAGEConv(hidden_channels, out_channels)

    def forward(self, x, edge_index):
        x = self.conv1(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.2, training=self.training)
        x = self.conv2(x, edge_index)
        return torch.sigmoid(x)

# =====
# 3. CORE FRAUD INTELLIGENCE ENGINE
# =====
class UniversalFraudEngine:
    def __init__(self, num_accounts=500):
        self.account_ids = [f"ACC_{i:04d}" for i in range(num_accounts)]
        self.acc_map = {acc: i for i, acc in enumerate(self.account_ids)}

        self.G, self.x, self.edge_index, self.acc_stats, self.ground_truth_ring, self.vip_acc =
self._init_network()
        self.account_risk_dict = self._train_gnn()

        high_risk = [acc for acc, score in self.account_risk_dict.items() if score > 0.4]
        subg = self.G.subgraph(high_risk).to_undirected()
        self.ring_members = set([node for c in nx.connected_components(subg) if len(c) >= 3 for
node in c])

    def _init_network(self):
        np.random.seed(42)
        sources = np.random.choice(self.account_ids, size=2000)
        targets = np.random.choice(self.account_ids, size=2000)
        mask = sources != targets
        df = pd.DataFrame({'source': sources[mask], 'target': targets[mask], 'amount':
np.random.exponential(100, size=sum(mask))})

        vip_account = "ACC_0001"
        vip_txs = [{'source': vip_account, 'target': np.random.choice(self.account_ids), 'amount':
np.random.uniform(15000, 25000)} for _ in range(15)]

        ring_accs = [f"ACC_{i:04d}" for i in range(10, 18)]
        mules = ["ACC_0018", "ACC_0019"]
        ring_txs = [{'source': r, 'target': m, 'amount': np.random.uniform(4900, 5000)} for r in
ring_accs for m in mules]

        df_all = pd.concat([df, pd.DataFrame(vip_txs), pd.DataFrame(ring_txs)], ignore_index=True)

        G = nx.DiGraph()
        for acc in self.account_ids: G.add_node(acc)
        for _, row in df_all.iterrows(): G.add_edge(row['source'], row['target'],
weight=abs(row['amount']))

        in_deg, out_deg, pagerank = dict(G.in_degree()), dict(G.out_degree()), nx.pagerank(G)

```

```

acc_stats, node_features = {}, []

for acc in self.account_ids:
    src_txs = df_all[df_all['source'] == acc]
    mean_amt = src_txs['amount'].abs().mean() if len(src_txs) > 0 else 50.0
    std_amt = src_txs['amount'].abs().std() if len(src_txs) > 1 else 20.0

    acc_stats[acc] = {'hist_mean': mean_amt, 'hist_std': std_amt if std_amt > 0 else 20.0}
    node_features.append([in_deg.get(acc, 0), out_deg.get(acc, 0), pagerank.get(acc, 0.0),
mean_amt, 0.0])

x = torch.tensor(node_features, dtype=torch.float)
edge_index = torch.tensor([[self.acc_map[src] for src in df_all['source']], [self.acc_map[dst]
for dst in df_all['target']]], dtype=torch.long)

return G, x, edge_index, acc_stats, ring_accs + mules, vip_account

def _train_gnn(self):
    model = FraudGNN(5, 16, 1)
    optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

    y = torch.zeros(len(self.account_ids), 1)
    for acc in self.ground_truth_ring: y[self.acc_map[acc]] = 1.0

    model.train()
    for _ in range(100):
        optimizer.zero_grad()
        out = model(self.x, self.edge_index)
        loss = F.binary_cross_entropy(out, y)
        loss.backward()
        optimizer.step()

    model.eval()
    with torch.no_grad():
        scores = model(self.x, self.edge_index).numpy().flatten()

    return {acc: float(scores[self.acc_map[acc]]) for acc in self.account_ids}

def process_user_file(self, input_file_path, export_excel_path="Fraud_Analysis_Output.xlsx"):
    tx_df = UniversalFileLoader.load_any_file(input_file_path)

    if tx_df.empty:
        print("⚠ Warning: No valid transactions detected in the provided file.")
        return tx_df

    evaluated = []
    for idx, row in tx_df.iterrows():
        tx_id = row['tx_id'] if row['tx_id'] not in ['N/A', 'UNKNOWN', ''] else f"TX_{idx+1:04d}"
        src, tgt, amt = row['source'], row['target'], row['amount']
        abs_amt = abs(amt)

        src_risk = self.account_risk_dict.get(src, 0.1)
        tgt_risk = self.account_risk_dict.get(tgt, 0.1)
        stats = self.acc_stats.get(src, {'hist_mean': 100.0, 'hist_std': 30.0})

        is_vip = (src == self.vip_acc) or (tgt == self.vip_acc)
        is_ring = (src in self.ring_members) or (tgt in self.ring_members)
        is_anomaly = (abs_amt > stats['hist_mean'] + 3 * stats['hist_std']) and (abs_amt > 1000)

```

```

risk_score = min(1.0, (src_risk * 0.4) + (tgt_risk * 0.3) + (0.4 if is_ring else 0.0) + (0.2 if
(is_anomaly and not is_vip) else 0.0)) * 100

```

```

reasons = []
if is_ring: reasons.append("Graph Fraud Ring matched")
if is_anomaly and not is_vip: reasons.append(f"Amount deviates from baseline ($
{stats['hist_mean']:.2f})")
if is_vip and not is_ring: reasons.append("Verified VIP baseline profile")

```

```

if is_ring or risk_score >= 80.0: action = "🔴 REJECT & FREEZE"

```

```

elif risk_score >= 45.0: action = "🟡 STEP-UP AUTH (MFA)"

```

```

else: action = "🟢 APPROVE TRANSACTION"

```

```

evaluated.append({
    'Tx_ID': tx_id,
    'Source': src[:35] + "..." if len(src) > 35 else src,
    'Target': tgt[:35] + "..." if len(tgt) > 35 else tgt,
    'Amount': f"₹{amt:.2f}",
    'Risk_Score': f"{risk_score:.1f}%",
    'In_Fraud_Ring': "YES" if is_ring else "NO",
    'Action': action,
    'Explanation': " | ".join(reasons) if reasons else "Matches normal profile"
})

```

```

results_df = pd.DataFrame(evaluated)

```

```

print("\n" + "="*120)

```

```

print("                                EVALUATION REPORT FOR YOUR TRANSACTIONS
")

```

```

print("="*120)

```

```

print(results_df[['Tx_ID', 'Source', 'Target', 'Amount', 'Risk_Score', 'In_Fraud_Ring', 'Action',
'Explanation']].to_string(index=False))

```

```

print("="*120 + "\n")

```

```

results_df.to_excel(export_excel_path, index=False)

```

```

print(f"✅ Excel Report generated and saved to: {export_excel_path}\n")

```

```

return results_df

```

```

# =====

```

```

# 4. INTERACTIVE FILE UPLOADER

```

```

# =====

```

```

def start_interactive_upload():

```

```

    try:

```

```

        from google.colab import files

```

```

        print("📁 Please select your transaction file (.xlsx, .csv, .pdf, .png, .jpg) to upload:")

```

```

        uploaded = files.upload()

```

```

        engine = UniversalFraudEngine()

```

```

        for filename in uploaded.keys():

```

```

            out_file = f"Evaluated_{filename.split('.')[0]}.xlsx"

```

```

            engine.process_user_file(filename, export_excel_path=out_file)

```

```

            files.download(out_file)

```

```

    except ImportError:

```

```

        file_path = input("Enter the path to your transaction file (e.g., AccountStatement.pdf): ").strip()

```

```

        engine = UniversalFraudEngine()

```

```

        engine.process_user_file(file_path, export_excel_path="Evaluated_Transactions.xlsx")

```

```

if __name__ == "__main__":

```

```

    start_interactive_upload()

```